

Project Interim Report

On

Music Playlist Management System (Using Core Java & OOP Concepts)

RU-100-01-00012

Object Oriented Programming With Java

SESSION: 2025-26



Submitted By

Student name

Md_Aarif

ERP-ID.RU-25-10804

**Bachelor of Technology in Computer Science &
Engineering with AI in association with Google**

Under the Guidance of

Mr. Soumik Karmakar

IBM SME and EXPERT

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

RUNGTA INTERNATIONAL SKILLS UNIVERSITY

BHILAI , CG

(April 2026)

◆ Abstract

The Music Playlist Management System is a Java-based console application designed to store, organize, and manage a collection of songs. The system enables users to maintain a dynamic playlist of songs, shuffle the playback order, and display the current queue in an organized manner.

This project demonstrates how real-world digital music players handle playlist management using simplified programming concepts. Manual tracking of song collections can be inefficient and error-prone; this system provides an automated way to store and process song information using object-oriented techniques.

The application allows users to add songs with their title and artist details, store multiple records using arrays of objects, shuffle the playlist using randomization logic, and display the queue using enhanced for-loops. It uses key object-oriented programming concepts such as encapsulation, aggregation (HAS-A relationship), constructors, and methods. Overall, the system simulates a basic music playlist manager and enhances understanding of Java OOP concepts.

◆ Introduction

The Music Playlist Management System is a console-based application developed using Java and object-oriented programming principles. It is designed to manage a collection of songs and simulate the behavior of a real-world digital music player including features like adding songs, displaying the playlist queue, and shuffling playback order.

In real-world scenarios, music playlist management plays an important role in streaming services, media players, and mobile applications. This project simulates such systems in a simplified way, helping users understand how song data is collected, organized, and processed programmatically.

The application provides features such as adding Song objects to a Playlist, displaying all stored songs, and randomly shuffling the playlist using Fisher-Yates shuffle logic. It also includes a menu-driven interface, making it user-friendly and easy to operate.

This project mainly focuses on applying core Java concepts like classes, objects, aggregation, arrays of objects, randomization, and enhanced for-loops to solve a real-life problem efficiently.

◆ Objectives

- To design and implement a Song class with encapsulated attributes (title and artist)
- To build a Playlist class that aggregates an array of Song objects
- To implement addSong() functionality for managing playlist entries
- To implement shufflePlaylist() using randomization algorithms
- To display the playlist queue dynamically using enhanced for-loops

- To apply object-oriented programming concepts such as encapsulation and aggregation
- To demonstrate HAS-A relationships between Playlist and Song classes

◆ Scope

This project is suitable for small-scale applications such as basic music collection management and educational purposes. It can be used in classrooms or personal projects to demonstrate OOP concepts using a relatable, real-world use case.

In the future, this system can be enhanced by integrating a database for large-scale song storage, adding a graphical user interface (GUI), connecting with real music streaming APIs, implementing additional features like search, delete, and favorites, and supporting persistence through file handling.

◆ System Requirements

Hardware Requirements:

- Computer/Laptop with minimum 4GB RAM
- Processor: Intel i3 or above

Software Requirements:

- Java JDK (Java Development Kit)
- IDE: VS Code / Eclipse / IntelliJ IDEA
- Operating System: Windows/Linux/Mac

◆ Methodology

The working of the system follows a step-by-step approach:

1. The program starts and creates a Playlist object.
2. Song objects are created with title and artist details and added to the playlist.
3. The system stores Song objects in an internal array inside the Playlist class.
4. The playlist queue is displayed using an enhanced for-loop, showing title and artist for each song.
5. The shufflePlaylist() method randomizes the order of Song objects in the array.
6. The shuffled playlist is displayed again to confirm the new playback order.
7. The program exits after completing all operations.

◆ Flowchart

[Flowchart diagram to be inserted here showing: Start → Create Playlist → Add Songs → Display Queue → Shuffle Playlist → Display Shuffled Queue → End]

◆ Classes - Methods / Variables Used

Class: Song

Variables:

- title (String)
- artist (String)

Methods:

- Constructor Song(String title, String artist)
- getTitle()
- getArtist()
- toString()

Class: Playlist

Variables:

- Song[] songs (Array of Song objects)
- int count (tracks number of songs added)

Methods:

- Constructor Playlist(int capacity)
- addSong(Song s)
- displayPlaylist()
- shufflePlaylist()

Class: Main (Driver Class)

Methods:

- main(String[] args) — Creates Playlist, adds Songs, displays and shuffles

◆ Implementation

The project consists of two main classes: Song and Playlist, along with a driver class containing the main() method.

The Song class is responsible for storing the title and artist of each song. It uses encapsulation by declaring variables as private and provides public getter methods to access them. A constructor initializes each Song object with its details.

The Playlist class demonstrates the HAS-A (aggregation) relationship by maintaining an internal array of Song objects. It provides the addSong() method to add songs to the collection, displayPlaylist() to print all songs using an enhanced for-loop, and shufflePlaylist() to randomly rearrange the songs using Fisher-Yates shuffle logic with the Random class.

The driver class creates a Playlist object, instantiates multiple Song objects, adds them to the playlist, displays the original order, calls shufflePlaylist(), and then displays the shuffled order. This demonstrates a complete, working implementation of object aggregation and randomization in Java.

◆ Source Code - Music Playlist Management App

Song.java

```
public class Song {
    private String title;
    private String artist;

    public Song(String title, String artist) {
        this.title = title;
        this.artist = artist;
    }

    public String getTitle() { return title; }
    public String getArtist() { return artist; }

    @Override
    public String toString() { return title + " - " + artist; }
}
```

Playlist.java

```
import java.util.Random;

public class Playlist {
    private Song[] songs;
    private int count;

    public Playlist(int capacity) {
        songs = new Song[capacity];
        count = 0;
    }

    public void addSong(Song s) {
        if (count < songs.length) {
            songs[count++] = s;
        } else {
            System.out.println("Playlist is full!");
        }
    }
}
```

```

    public void displayPlaylist() {
        int num = 1;
        for (Song s : songs) {
            if (s != null)
                System.out.println(num++ + ". " + s);
        }
    }

    public void shufflePlaylist() {
        Random rand = new Random();
        for (int i = count - 1; i > 0; i--) {
            int j = rand.nextInt(i + 1);
            Song temp = songs[i];
            songs[i] = songs[j];
            songs[j] = temp;
        }
    }
}

```

Main.java

```

public class Main {
    public static void main(String[] args) {
        Playlist playlist = new Playlist(5);

        playlist.addSong(new Song("Believer", "Imagine Dragons"));
        playlist.addSong(new Song("Shape of You", "Ed Sheeran"));
        playlist.addSong(new Song("Blinding Lights", "The Weeknd"));
        playlist.addSong(new Song("Levitating", "Dua Lipa"));
        playlist.addSong(new Song("Stay", "The Kid LAROI"));

        System.out.println("Playlist:");
        playlist.displayPlaylist();

        System.out.println("\nShuffling Playlist...");
        playlist.shufflePlaylist();

        System.out.println("\nShuffled Playlist:");
        playlist.displayPlaylist();
    }
}

```

◆ Sample Input / Output

```

Playlist:
1. Believer - Imagine Dragons
2. Shape of You - Ed Sheeran
3. Blinding Lights - The Weeknd
4. Levitating - Dua Lipa
5. Stay - The Kid LAROI

Shuffling Playlist...

Shuffled Playlist:
1. Blinding Lights - The Weeknd
2. Stay - The Kid LAROI
3. Believer - Imagine Dragons

```

4. Shape of You - Ed Sheeran
5. Levitating - Dua Lipa

◆ Gantt Chart

Task	Duration
Planning & Design	1 Day
Coding (Classes & Logic)	2 Days
Testing	1 Day
Documentation	1 Day
Total Estimated Time	5 Days

◆ Conclusion

The Music Playlist Management System project successfully demonstrates the use of Java and object-oriented programming concepts in solving a real-world problem. It provides a functional system to store, organize, shuffle, and display songs efficiently using object aggregation.

The project helped in understanding key concepts such as encapsulation, the HAS-A relationship, arrays of objects, enhanced for-loops, and randomization algorithms. It also improved logical thinking and modular design skills.

Overall, this system serves as a strong foundation for developing more advanced music management applications and demonstrates how real-world digital media systems can be modeled using fundamental OOP principles in Java.